Tallymark: catching what the extraction missed

Design document · RadHires Harness Challenge · Pedro Hang · October 2026

> **In one sentence:** after a Bill of Characteristics has been built from an engineering drawing, Tallymark reads the drawing again on its own, greys out everything a row already covers, and leaves lit only the requirements nobody listed, so the quality engineer checks about 4 items instead of hunting through 200.

1. The problem thesis

Who has it. The quality engineer at a manufacturer that machines parts for aerospace and other regulated customers. Before the first production part ships, the AS9102 standard requires a *First Article Inspection Report*. The customer sends the drawing; the supplier lists every requirement on it (one row per callout or note, the "Bill of Characteristics" on Form 3), measures the first part against every row, and submits the report for approval.

What it costs. If a single requirement on the drawing has no row, that feature is never measured and the customer can reject the whole report. That means re-inspection, re-submission, a late shipment and a quality mark against the supplier. Today the engineer proves "nothing was missed" by eye, across hundreds of callouts and several sheets; a common manual trick is covering each extracted item with white-out until only the misses remain. AI extraction tools now draft the table faster, but they make checking it *more* important, not less: the engineer signs for a table they did not type. (I give no dollar figure: I could not find a verified public number and did not want to invent one.)

Why an AI harness fits. The job is a cross-check between two views of the same truth: what the drawing says and what the table says. Most of it is deterministic (find the text, match it to a row, ignore the title block), so code should do it. What is left is a small, genuinely ambiguous residue, such as "is this bare number a requirement or just a label?", where a vision model is good and cheap. A harness keeps the model on that residue only, and makes every flag explainable.

Where it is not a fit. It catches what was *left out*, not values that were *misread* (12.5 typed as 125). It cannot flag text the OCR never sees. It does not replace the engineer's signature: it points, the human decides.

2. The end user and the interface

Who. A quality engineer or inspector: fluent in drawings and tolerance symbols, not a software person, working a queue under delivery pressure, personally accountable for the report. They want a short, trustworthy list, not a chat about their drawing.

What they do.

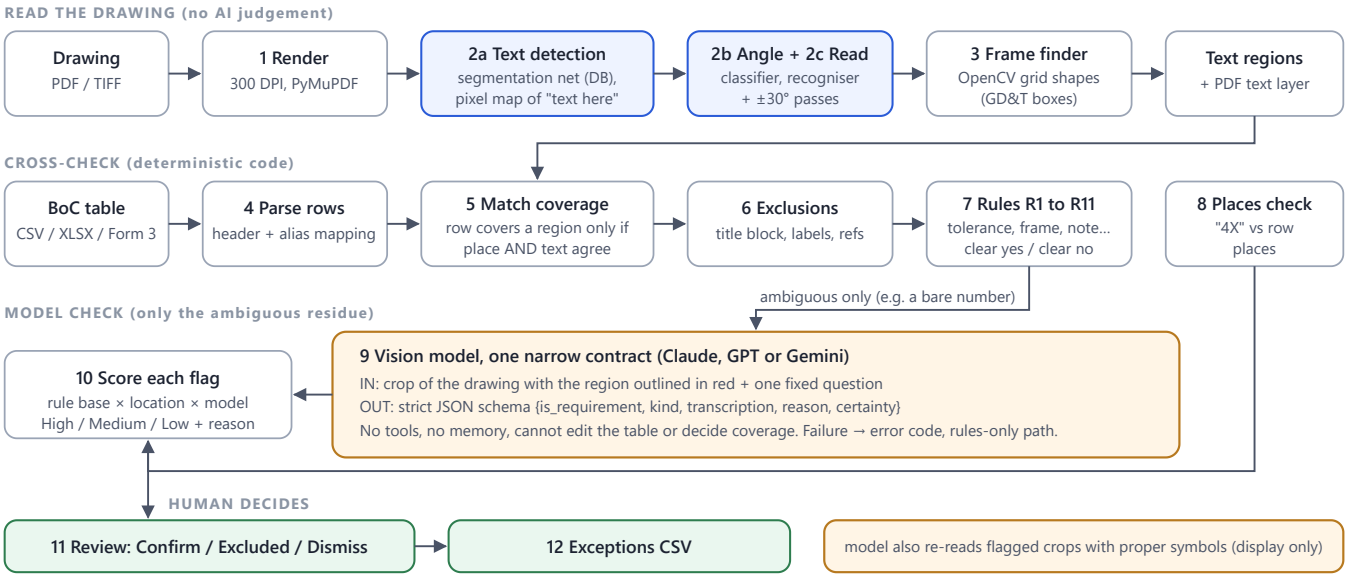
- **Load** the drawing (PDF/TIFF) and the table (CSV/XLSX, including the official Form 3 layout), or try the bundled sample. Click **Run check**.
- **See** the drawing faded, with covered regions tinted grey and only the gaps lit: `U` requirement with no row, `P` "4X" callout whose row lists fewer places.
- **Decide** each flag: Confirm miss, Excluded by customer rule, or Dismiss (keys C / E / D, undo always available).
- **Export** a small exceptions CSV to fix the table.

How they know it worked. A big count ("4 unaccounted, 0 of 4 reviewed") that turns green when done; an *Original* toggle to prove nothing was painted over; every flag says which rule fired and how confident it is; hovering a grey region shows which row covers it.

What happens when it does not. A bad file gets a plain message in the drop zone. A sheet with no rows gets a warning before the run. If the AI model is unavailable (no key, bad key, rate limit, outage) the run still completes on rules only, a banner says why, and the ambiguous items are kept as low-confidence flags instead of silently dropped. Simulated data is labelled on screen.

Why a web app, not a chat. The work is visual and spatial, and the habit it replaces (white-out on a printout) is visual too. The output is a decision per flag, not a conversation; a chat window would hide the drawing. A browser needs no install on a locked-down shop-floor PC. For first-time users: a plain explainer, an animated preview of a run, hover help on each input, a guided tour, dark mode and an interface-size control.

3. Architecture and the harness



Blue: small local neural networks (OCR). Amber: the only calls to a large vision model. Green: the person. Everything else is plain, testable code.

How it works. The drawing is rendered and read by three small OCR networks that run locally (PaddleOCR models via RapidOCR): a **text detector**, which is a segmentation model that marks every pixel that looks like text and draws boxes around it; an **angle classifier**; and a **recogniser** that turns each box into characters. Tilted isometric callouts get extra passes at ±30°. OpenCV finds tolerance frames by their grid shape, so a frame counts even when its symbols are unreadable. The table is parsed separately. A row "covers" a drawing region only if *both* its location and its text agree, because callouts sit side by side and a location-only match would let a neighbour hide the one real miss. Title blocks, labels and reference dimensions are excluded. Eleven readable rules decide the clear cases. Only what is still ambiguous goes to the model.

The harness controls the model. The model is one component the harness drives, not the driver. The harness decides *when* it is called (only for leftovers the rules cannot settle), *what* it sees (a crop with the region outlined in red, never the whole sheet or the table), *how* it must answer (one fixed question, a strict JSON schema: is_requirement, kind, transcription, reason, certainty) and *what* its answer may change (keep or drop an ambiguous flag, with a stated reason; never write to the table or decide coverage). Every answer is schema-validated, and any failure falls back to rules with the item kept for review. A second call re-types flagged text with proper symbols, for display only. The same contract runs on Anthropic, OpenAI or Google. I chose this bounded contract over a free-roaming agent on purpose: in a tool whose whole job is to miss nothing, every model decision must be scoped, explainable and auditable.

Failure handling

- Every provider error (bad key, 403, 404, rate limit, network, malformed output) becomes a short code; the run never crashes and falls back to rules-only with a banner.
- Inputs are validated with plain messages (encrypted PDF, >20 sheets, missing columns, Form 3 header under a title block).
- Low confidence is shown, not hidden: "model check skipped, review it".
- OCR is cached per drawing, so re-runs take about a second.

Knowing whether it is any good

An eval harness plants misses (deleted rows, wrong "places") into public NIST test drawings and scores the output. 27 unit tests cover parsing, matching, request shapes for all three providers and the failure paths.

Rules only, model off	Demo drawing	8 unseen drawings
Planted misses found	100% (8/8)	98% (63/64)
Flags that were real	100%	34%
False flags per clean sheet	0	2.78 (0.48 excl. ambiguous)

The demo drawing was used for tuning, so it is not held out. The unseen set's noise is mostly garbled stacked tolerances: exactly the ambiguous items the model check exists to judge, which I have not yet measured.

4. Tooling and tradeoffs

Choice	Why	What I weighed against it
Rules first, model on the residue	Explainable flags; cheap; testable; one wrong flag costs trust.	One big "find what's missing" prompt over the whole sheet: simpler to build, but unverifiable, costly per sheet, and GD&T reading by large models is not well benchmarked.
Local OCR (RapidOCR / PaddleOCR on ONNX)	Free, offline, deterministic; customer drawings never leave the machine for this step.	Cloud document AI (Azure, Google): stronger on messy scans, but cost, data-handling questions for export-controlled drawings, and a network dependency.
Provider-agnostic structured calls	Same schema on Claude, GPT or Gemini; the shop uses whichever key it already has.	Locking to one vendor: less code, but a procurement blocker and a single point of failure.
FastAPI + vanilla JS, no build step	Runs with one command; easy for a reviewer to read end to end.	React / Next: nicer component model, but heavier tooling for a one-screen workflow.
Not built	A tool-using agent loop, a vector store / RAG, fine-tuning, user accounts. None of them moves the core metric (a missed row caught before submission) for this slice.	

5. Reflections

Time spent. [fill in: hours on this submission] , on top of an earlier prototype.

How I worked. Iteratively, one step at a time, with Claude Code as a pair: I used skills like *grill-me* to have the design interrogated before building, then tightened the pieces one by one (provider fallback, Form 3 parsing, the review UI). I picked this problem because I had already explored it a couple of months earlier, and engineering drawings are a natural fit for today's models: they are visual, dense, and full of structured meaning that used to need a person to transcribe.

Why now. AI extraction is only starting to show up in inspection and ballooning tools, so adoption in this field is early and growing. Generative vision models can now read messy visuals and return structured data cheaply, which moves the bottleneck from *typing* the table to *trusting* it. That trust step is where this harness sits.

Next, with more time.

- Measure the model check on the unseen drawings and tune its threshold against the 34% precision.
- Test with a real extractor's export instead of a simulated one.
- A second layer for misread values (row says 12.5, drawing says 125).
- Feed the engineer's decisions back into exclusions and rules.
- Use CAD product data (STEP AP242) as ground truth where customers supply it.

What I am least sure about.

- Precision on real customer drawings: unseen-drawing noise is the main risk to adoption.
- Whether real exports carry balloon coordinates; without them matching falls back to text and is weaker.
- OCR on poor scans and dense GD&T; whatever OCR misses, the tool cannot flag.
- Whether the model check actually removes the noise. I designed for it but have not measured it yet.